SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5116

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage : 20%

QUESTIONS: **Total Marks: 50**

Annexure A

Scenario-Based

Code of Conduct:

I J.Vishwanth (192573030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

J.Vishwanth

Signature of the student:

Question 4: Optimization and Security Analysis of Diffie–Hellman Key Exchange

Introduction

Diffie–Hellman (DH) is a public-key key-exchange method that allows two users to establish a common secret over an insecure network. The secret is not sent directly. Instead, each user exchanges a public value and independently calculates the same shared key.

A. Debug incorrect shared-key generation due to improper modular arithmetic

Let p be a large prime number and g a generator modulo p. Alice chooses a private value a and Bob chooses a private value b. Alice calculates $A = g^a \bmod p$, and Bob calculates $B = g^b \bmod p$. Alice computes $K = B^a \bmod p$, while Bob computes $K = A^b \bmod p$. Both obtain the same value because $(g^b)^a \bmod p = (g^a)^b \bmod p = g^{ab} \bmod p$.

Common implementation mistakes include omitting the modulus, applying it at the wrong stage, using ordinary division instead of modular arithmetic, and confusing private values with public values. Use modular exponentiation rather than calculating a huge power first.

Correct Python implementation

```
p = 23
g = 5
a = 6 # Alice's private key
b = 15 # Bob's private key

A = pow(g, a, p) # Alice's public value
B = pow(g, b, p) # Bob's public value

alice_key = pow(B, a, p)
bob_key = pow(A, b, p)

assert alice_key == bob_key
print('Shared key:', alice_key)
```

The three-argument form pow(base, exponent, modulus) performs modular exponentiation efficiently. The small values shown are for demonstration only; real applications need trusted parameters and public-value validation.

B. Optimize exponentiation operations for large prime values

For large primes, direct evaluation of g**a creates an extremely large intermediate integer and uses unnecessary time and memory. Fast modular exponentiation, such as square-and-multiply, processes the exponent bit by bit, repeatedly squares the current value, and reduces intermediate results modulo p. This makes the calculation much more practical.

In Python, pow(g, a, p) already uses an optimized implementation, so it is preferable to (g ** a) % p. Avoid regenerating parameters unnecessarily, keep private values out of logs, and use a well-maintained cryptographic library in production systems.

Question 4 (continued)

C. Mitigate man-in-the-middle attacks and evaluate performance impact

Basic Diffie–Hellman does not authenticate the communicating parties. An attacker positioned between Alice and Bob can establish one key with Alice and a different key with Bob, then relay messages while reading or modifying them. This is called a man-in-the-middle attack.

To mitigate this risk, authenticate the exchange using digital signatures and certificates, or use an authenticated key-exchange protocol such as TLS with ephemeral Diffie–Hellman. The parties should verify identity, validate received public values, and derive encryption keys from the shared secret with a standard key-derivation function instead of using the raw secret directly.

Authentication and validation add some computation and communication overhead. However, efficient modular exponentiation keeps the key exchange practical, and the added cost is justified because it helps prevent impersonation. Ephemeral DH can also provide forward secrecy when session private values are generated securely and discarded after use.

Additional implementation checks

1. Confirm that both parties use the same approved prime p and generator g.
2. Keep private exponents secret and generate them with a secure random source.
3. Check received public values according to the chosen group and protocol.
4. Do not use the shared secret directly as an encryption key; apply a standard key-derivation function.
5. Prefer established cryptographic libraries and protocols instead of custom cryptographic code.

Conclusion

Correct modular arithmetic ensures that both parties derive the same shared key. Efficient modular exponentiation improves performance for large primes, while authentication, public-value validation, and secure key derivation address major security risks. These measures improve reliability and security, although production systems should use established cryptographic libraries rather than a homemade protocol.